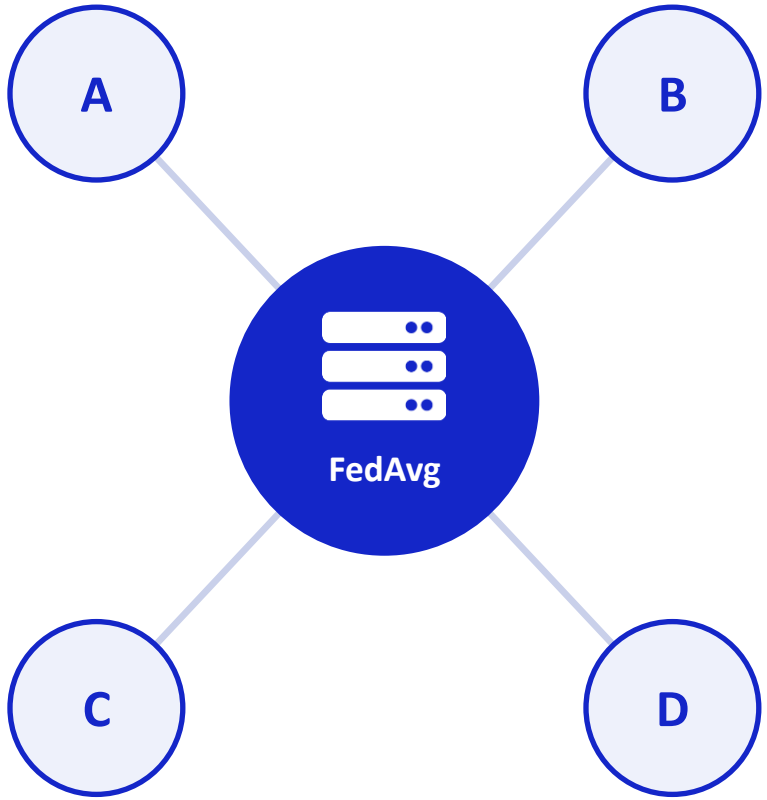
PS-003

Federated Traffic-Flow Prediction with Communication-Cost Evaluation



Team NEUROVIA

Category: Smart Cities • Dataset: PEMS-BAY

AIORI Hackathon 3

October 13, 2026

Problem Description

Edge traffic forecasting is costly in bandwidth and privacy.



Bandwidth cost

Centralised forecasting streams raw readings from every roadside sensor to a cloud server over cellular links.



Privacy exposure

Raw logs reveal private travel patterns, so sharing only model weight updates is attractive.



Non-IID data

Each intersection sees different traffic, which can slow federated convergence and hurt accuracy.

The challenge

Compare federated and centralised traffic-flow prediction on a public dataset split across simulated intersections, and measure how non-IID data and communication overhead affect accuracy and convergence.

In scope

3–5 simulated clients, FedAvg, MAE, bytes per round, 3 seeds

Out of scope

Live cameras, city-wide deployment, formal privacy guarantees

Background

Traffic forecasting, federated averaging and why communication matters.

1 Traffic forecasting

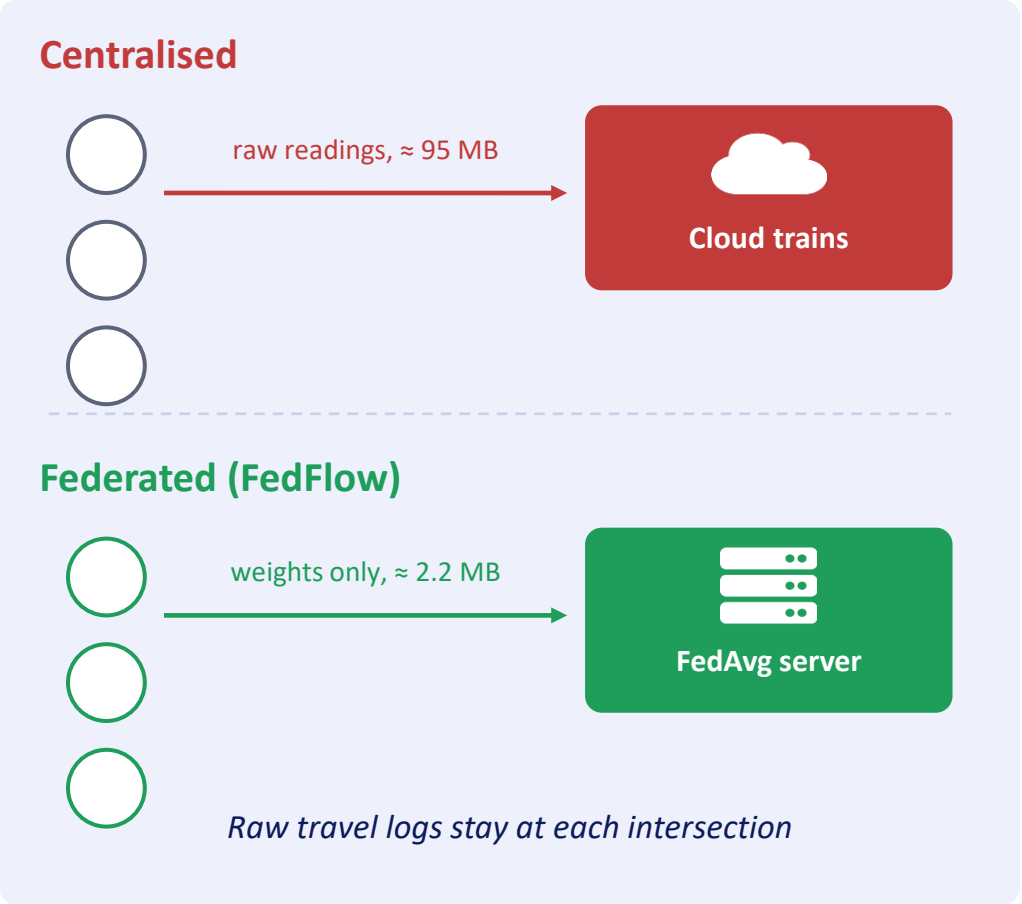
Predict near-future traffic from recent sensor history. PEMS-BAY provides 325 road sensors at 5-minute resolution [4].

2 Federated averaging

Clients train locally and the server averages their weights, weighted by data size n_k / n. Raw data never moves [1].

3 Non-IID and cost

Differing client distributions cause client drift [3]. Every round of weight uploads is a communication cost [2].



Literature & RFC Review

Only the most relevant work and standards are covered.

Work	Key contribution	Use in FedFlow
McMahan et al., 2017 [1]	Introduces FedAvg: local training, weighted weight averaging	Core training algorithm
Konečný et al., 2016 [2]	Strategies to cut federated upload cost, such as compressed updates	float16 uploads, fewer rounds
Zhao et al., 2018 [3]	Shows non-IID data degrades FedAvg accuracy	Explains why our location-based split is non-IID
Li et al., 2018 [4]	Defines PEMS-BAY traffic benchmark with 325 sensors	Dataset and chronological split
IEEE 3652.1-2020 [5]	Architecture guide for federated machine learning (no IETF RFC covers FedAvg)	Client / server role design

Proposed Solution

FedFlow: a federated simulator with built-in communication accounting.

1

Load and clean

PEMS-BAY, 325 sensors; fill missing readings

2

Chronological split

70:30 train/test, no leakage across time

3

Build windows

12 past steps to forecast +30 minutes

4

Virtual clients

3 intersections of 109, 108, 108 sensors

5

FedAvg rounds

E local epochs, weighted by n_k / n

6

Byte accounting

Bytes uploaded and downloaded per round

7

Evaluate

MAE over 3 seeds vs baseline

Models

- Linear: 114 weights
- MLP 114→64→32→1: 9,473 parameters
- Centralised linear baseline

Optimization Proposed

Before and after: where FedFlow saves bytes and protects accuracy.

Aspect	Naive federated setup	FedFlow
Upload size	float32 weights	float16 uploads, half the bytes per upload
Local work	1 epoch, many rounds	1 / 2 / 5 epochs compared, fewer rounds
Aggregation	Plain average of clients	Sample-weighted average (n_k / n)
Missing readings	Gaps feed the model	Forward and backward fill
Reporting	Error per round only	Error vs cumulative bytes sent

Uplink data per training run

≈ 95 MB

raw sensor data

↓

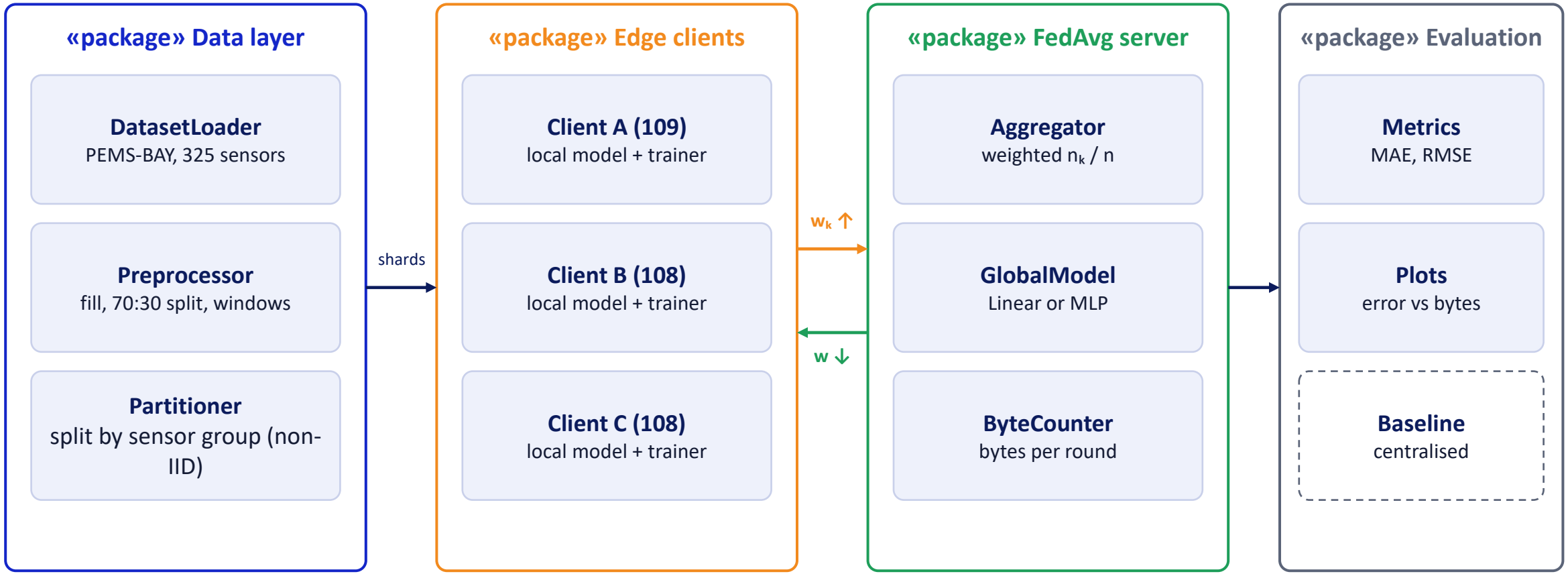
≈ 2.2 MB

model weights only

Estimated from model size and data volume; computed from model size and rounds..

Solution Architecture & Design

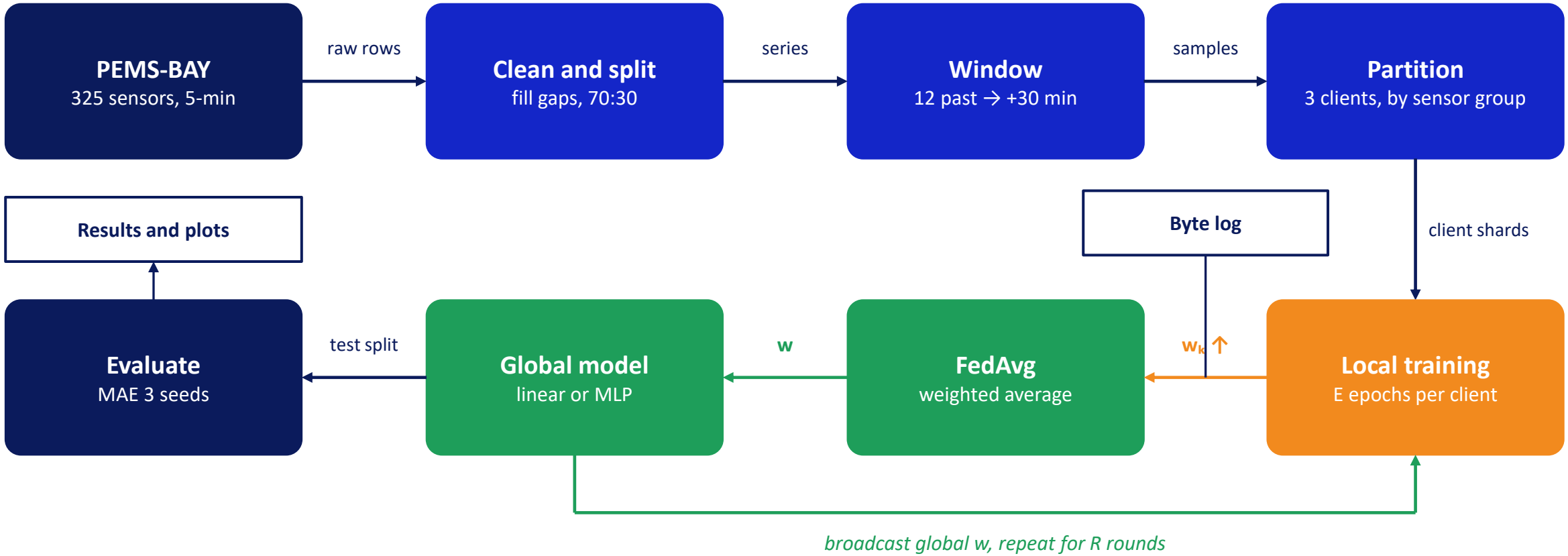
Component view: data layer, edge clients, FedAvg server and evaluation.



w_k = local weights from client k (uplink) w = global weights (downlink) dashed = baseline reference

Data Flow Diagram

From raw sensor readings to byte-versus-error results.



Raw data never leaves the clients; only weights cross the network.

Deployment Timeline

Six milestones, each producing a checkable deliverable.



Step 1

Data prep

Load PEMS-BAY, fill gaps, chronological 70:30 split

Step 2

Baseline

Centralised linear model, MAE and RMSE

Step 3

Simulator

FedAvg with 3 clients; linear and MLP

Step 4

Experiments

float16, 1/2/5 epochs, 3 seeds. Future work: controlled IID vs non-IID comparison.

Step 5

Analysis

Error vs bytes, per-client error, final report

Step 6

Hand-over

Private repo, aiori-hackathon as collaborator

Methodology: build, run on 3 random seeds, log bytes and errors every round, then compare federated against centralised. Code is pushed to the private GitHub repo for mentor oversight.

Bibliography

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Aguera y Arcas. “Communication-Efficient Learning of Deep Networks from Decentralized Data.” In: AISTATS, PMLR 54 (2017), pp. 1273–1282.

[2] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, D. Bacon. “Federated Learning: Strategies for Improving Communication Efficiency.” arXiv:1610.05492 (2016).

[3] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra. “Federated Learning with Non-IID Data.” arXiv:1806.00582 (2018).

[4] Y. Li, R. Yu, C. Shahabi, Y. Liu. “Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting.” In: ICLR (2018). arXiv:1707.01926.

[5] IEEE Std 3652.1-2020. “Guide for Architectural Framework and Application of Federated Machine Learning.”

[6] A. Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” In: NeurIPS (2019).

Thank You for Your Attention!

PS-003 • Federated Traffic-Flow Prediction with Communication-Cost Evaluation

3	3	2	325
virtual intersections	random seeds	models: linear and MLP	PEMS-BAY sensors

Team NEUROVIA • AIORI Hackathon 3